

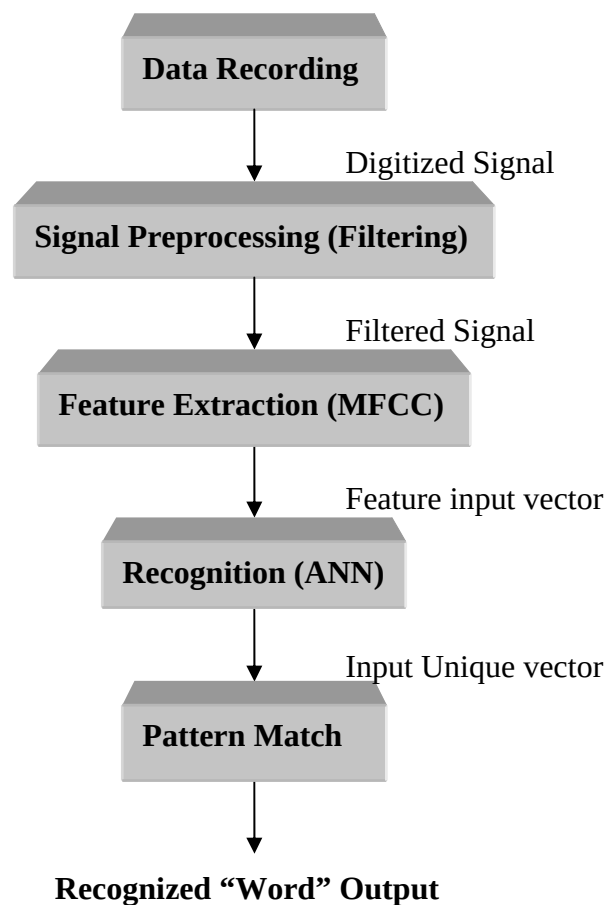
### บทที่ 3

#### การออกแบบและพัฒนา

#### 3.1 บทนำ

การทำงานของระบบในปริิณญานิพนธ์ประกอบด้วย 3 ส่วนหลัก คือ

- การประมวลผลเบื้องต้น (Preprocessing)
- การสกัดค่าลักษณะสำคัญ (Feature extraction)
- การรู้จำ (Recognition)



รูปที่ 3.1 แสดงองค์ประกอบหลักของระบบรู้จำเสียง

### 3.2 โปรแกรมที่ใช้ในระบบ

โปรแกรมทั้งที่ใช้ในระบบประกอบไปด้วย

#### 3.2.1 ส่วนการอัดเสียง

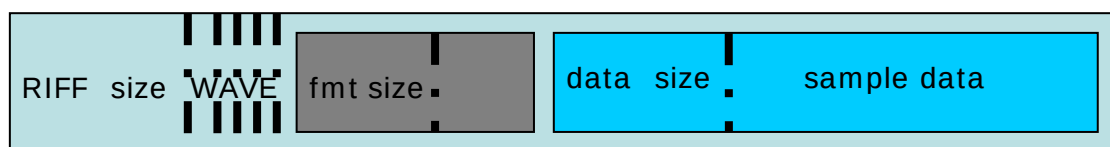
ทำโดยใช้โปรแกรม Sound Recorder ที่มีมาพร้อมกับ Windows XP แล้ว



รูปที่ 3.2 แสดงการใช้งานโปรแกรม Sound Recorder

ในการบันทึกเสียงพูดเป็นข้อมูลดิจิทัลโดยใช้การ์ดเสียง จะทำให้ได้เวฟไฟล์ ( .wav ) ที่มีรูปแบบของไฟล์ RIFF ( Resource International File Format ) ซึ่งเป็นรูปแบบมาตรฐานของ MS windows ที่ใช้กันอย่างกว้างขวาง

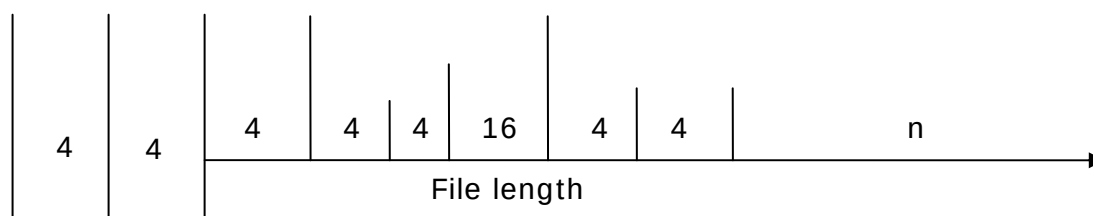
ไฟล์ RIFF นั้นมีโครงสร้างเชิงซ้อนคือ มีลักษณะเป็นกลุ่ม ( Chunk ) ซึ่งภายในกลุ่มนี้ยังประกอบด้วยกลุ่มย่อยซึ่งมีโครงสร้างคล้ายกัน ดังแสดงในรูปที่ 3.3



RIFF Chunk

Fmt Data Chunk

Sample Data Chunk



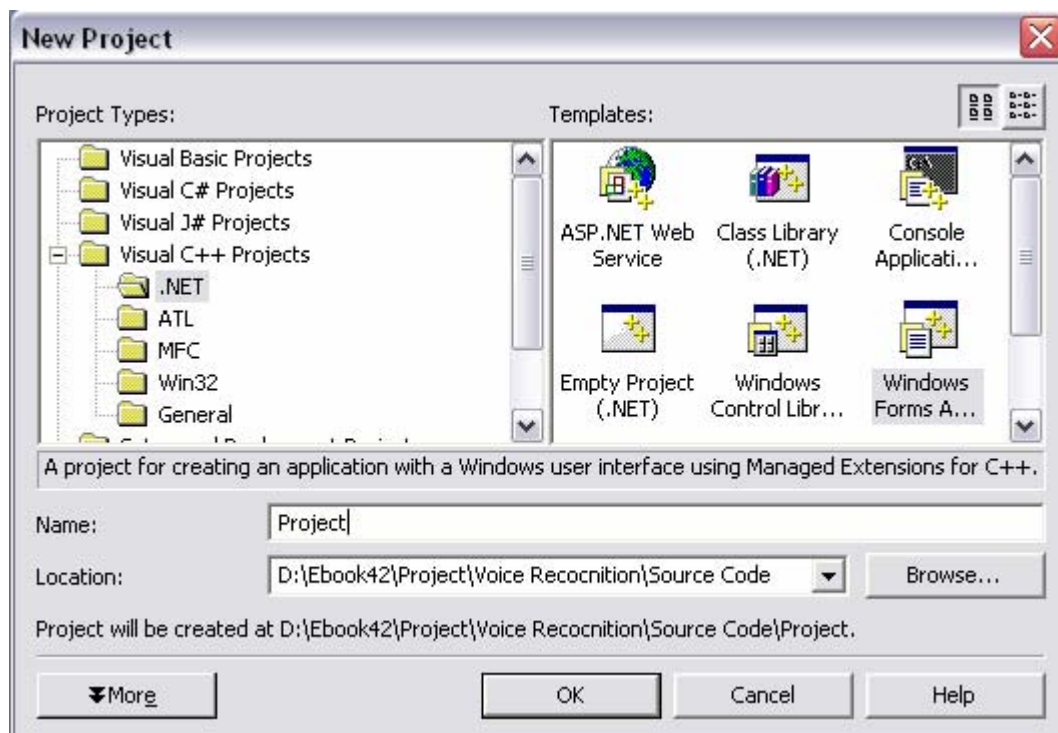
รูปที่ 3.3 แสดงโครงสร้างเวฟไฟล์ที่มีรูปแบบของไฟล์ RIFF

จากรูปที่ 3.3 จะเห็นว่า กลุ่ม RIFF ( RIFF Chunk ) จะเป็นกลุ่มใหญ่ที่สุด ซึ่งประกอบด้วย กลุ่มรูปแบบของข้อมูล ( Format Data Chunk ) และกลุ่มข้อมูลจากการแซมปลิง ( Sample Data Chunk )

ในการทดลอง กำหนดให้เก็บเสียงในรูปแบบของไฟล์ WAV อัตราการชักตัวอย่าง (Sampling rate) ที่ 11.025 กิโลเฮิร์ต ตัวอย่างละ 16 บิต และแบบช่องสัญญาณเดียว(Mono)

### 3.2.2 ส่วนประมวลผลและพัฒนา

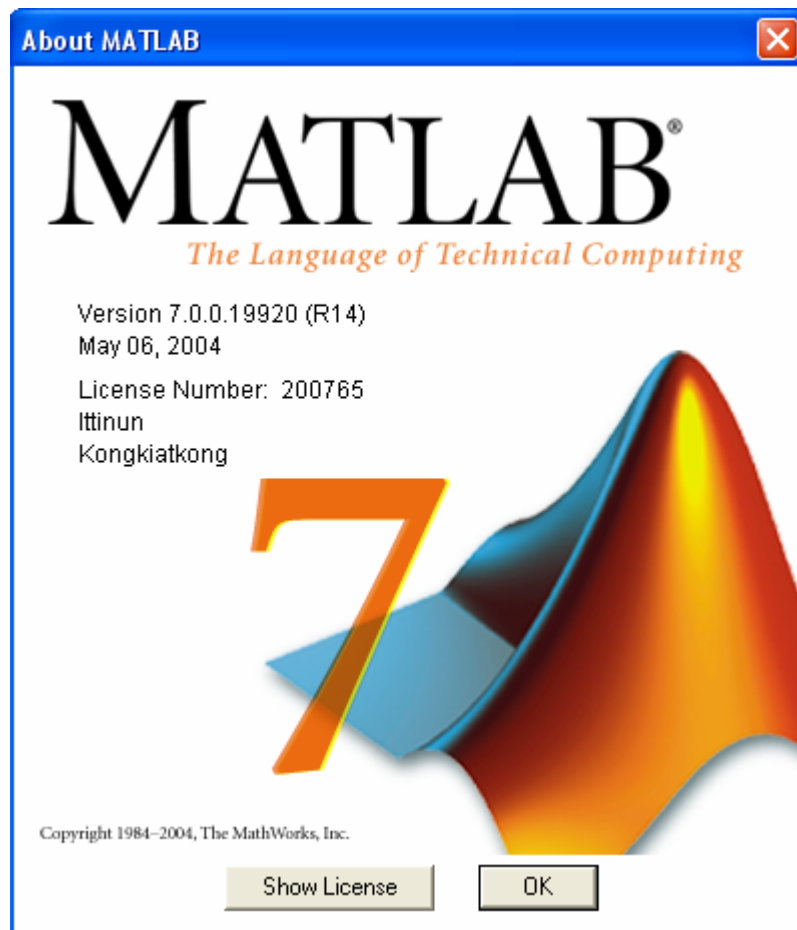
ทำการประมวลผลและพัฒนา พัฒนาด้วยโปรแกรม Microsoft Visual C++ .NET พัฒนาด้วยรูปแบบ C# เพื่อให้สะดวกต่อการนำไปรวมกับส่วนโปรแกรมที่เกี่ยวข้องโดยออกแบบให้ส่วนนี้เป็นคลาส แล้วเรียกใช้ผ่านเมธอดของออบเจ็กแทนการเรียกใช้แบบฟังก์ชัน ทำให้การแก้ไขและพัฒนาโมดูลทำได้ง่ายและสะดวก



รูปที่ 3.4 แสดงการใช้งาน Microsoft Visual C++ .NET แบบ Windows Forms Application

### 3.2.3 ส่วนแสดงผล

ทำการทดสอบทฤษฎีและการทำงานโดยใช้ MATLAB 7.0 เนื่องจากมีฟังก์ชันสำหรับการแปลงสัญญาณเสียง มีกราฟแสดงรูปแบบของสัญญาณเสียง และง่ายต่อการเรียนรู้และใช้งาน



รูปที่ 3.5 แสดงการใช้งาน MATLAB 7.0

### 3.3 การประมวลผลเบื้องต้น (Preprocessing)

สัญญาณเสียงที่ผ่านการแปลงสัญญาณเป็นดิจิทัลแล้ว จะนำมาผ่านขั้นตอนการประมวลผลเบื้องต้น ซึ่งประกอบด้วยขั้นตอนต่างๆ ดังนี้

#### 3.3.1 การกรองทางความถี่ (Filtering)

เป็นขั้นตอนในการกรองสัญญาณในช่วงความถี่ที่ไม่ต้องการออกโดยอาศัยตัวกรองแบบดิจิทัล โดยใช้ตัวกรองแบบดิจิทัลชนิดผ่านความถี่สูง กำหนดจุดผ่านความถี่ (Cutoff frequency) ที่ 200 เฮิรตซ์ เพื่อป้องกันสัญญาณรบกวนความถี่ต่ำที่เกิดจากแหล่งกำเนิดไฟฟ้า

#### 3.3.2 การตัดหัว-ท้ายเสียง (Endpoint detection)

เป็นขั้นตอนในการกำหนดจุดเริ่มต้นและจุดสิ้นสุดของเสียงโดยการแยกส่วนที่เป็นคำพูดออกจากส่วนที่ไม่ใช่คำพูด วิธีในการตัดหัว-ท้ายเสียงหลายวิธี เช่น ใช้ค่าระดับพลังงาน (Energy level) ใช้อัตราการตัดศูนย์ (Zero-crossing rate) เป็นต้น

#### 3.3.3 การนอร์มอลไลซ์ทางเวลา (Time normalization)

เป็นขั้นตอนการเพิ่มหรือลดขนาดความยาวของสัญญาณในเชิงเวลา เพื่อปรับแต่งขนาดความยาวของสัญญาณให้เหมาะสมตามต้องการทั้งนี้จะขึ้นอยู่กับกระบวนการในการรู้จำว่าเป็น

ต้องทำการนอร์มอลไลซ์สัญญาณให้เท่ากันหรือไม่ วิธีในการนอร์มอลไลซ์ทางเวลามีหลายวิธี เช่น การเปลี่ยนอัตราการซิกตัวอย่าง (Sampling rate changing) การประมาณค่าในช่วงเชิงเส้น (Linear interpolation) และการเหลื่อมและรวมส่วนย่อยแบบซิงโครไนซ์ (Synchronized overlap-and-add) เป็นต้น

### 3.4 การสกัดค่าลักษณะสำคัญ (Feature)

การสกัดค่าลักษณะสำคัญ คือการวิเคราะห์หาค่าที่จะใช้แทนสัญญาณเสียง เพื่อนำไปใช้ในขั้นตอนการรู้จำ แบ่งได้เป็น 3 กลุ่มหลัก กลุ่มแรกเป็นค่าลักษณะสำคัญระดับสูง (High level feature) ได้แก่ สำเนียงการพูด รูปแบบในการพูด และความเร็วในการพูด เป็นต้น ในกลุ่มที่สอง จะใช้ค่าลักษณะสำคัญทางฉันทลักษณ์ (Prosodic feature) เช่น ค่าความถี่มูลฐาน (Fundamental frequency) ความถี่ฟอร์แมนท์ (Formant frequency) และระดับพลังงาน (Energy profile) เป็นต้น ถึงแม้ว่าค่าลักษณะสำคัญแบบนี้จะมีประสิทธิภาพสูงในการรู้จำ แต่ยากในการสกัดจากสัญญาณ กลุ่มสุดท้าย เรียกว่าค่าลักษณะสำคัญแบบเอนเวโลปของสเปกตรัม (Spectral envelop feature) เป็นกลุ่มที่นิยมใช้กันมาก เนื่องจากค่าลักษณะสำคัญส่วนใหญ่สำหรับการรู้จำเสียงจะรวมอยู่ในข้อมูลเชิงสเปกตรัมนี้ อีกทั้งยังง่ายและสะดวกในการคำนวณหาค่าด้วย ตัวอย่างค่าลักษณะสำคัญแบบนี้ได้แก่ สัมประสิทธิ์การประมาณพหุเชิงเส้น (Linear prediction coefficients: LPC), สัมประสิทธิ์เซปสตรัม (Cepstral coefficient) และพัฒนาการอีกมากมายจากเซปสตรัมปกติ อาทิเช่น สัมประสิทธิ์เซปสตรัมบนสเกลเมล (Mel frequency cepstral coefficients: MFCC) เซปสตรัมแบบหักลบค่าเฉลี่ย (Cepstral mean subtraction: CMS) และเซปสตรัมแบบผ่านตัวกรองภายหลัง (Post filtered cepstrum: PFL) เป็นต้น นอกจากนี้ ยังมีการคำนวณค่าการเปลี่ยนแปลง (Derivative หรือ Delta) ของสัมประสิทธิ์เหล่านี้มาใช้เป็นค่าลักษณะสำคัญเพิ่มเติมได้ด้วย

สำหรับการคำนวณค่าลักษณะสำคัญแบบเอนเวโลปของสเปกตรัมจะมีขั้นตอนดังนี้

#### 3.4.1 การเน้นสัญญาณขั้นต้น (Preemphasis)

เป็นขั้นตอนในการบีบอัดสัญญาณเสียงโดยนำสัญญาณเสียงผ่านตัวกรองลำดับหนึ่ง (First-order filter) ซึ่งจะเพิ่มอัตราส่วนสัญญาณต่อสัญญาณรบกวน (Signal to noise ratio)

#### 3.4.2 การแบ่งเป็นส่วนย่อย (Frame)

เป็นขั้นตอนในการแบ่งสัญญาณเสียงเป็นส่วนย่อย ขนาดความยาวประมาณ 10 – 40 มิลลิวินาที ซึ่งทำให้สัญญาณเสียงมีคุณสมบัติเปลี่ยนแปลงตามเวลาน้อยมาก หรือไม่มีเลย เพื่อให้สามารถสร้างแบบจำลองการกระจายของหน่วยสัญญาณเสียงย่อยทางสถิติได้

#### 3.4.3 การลดขอบด้วยฟังก์ชันหน้าต่าง (Smoothing window)

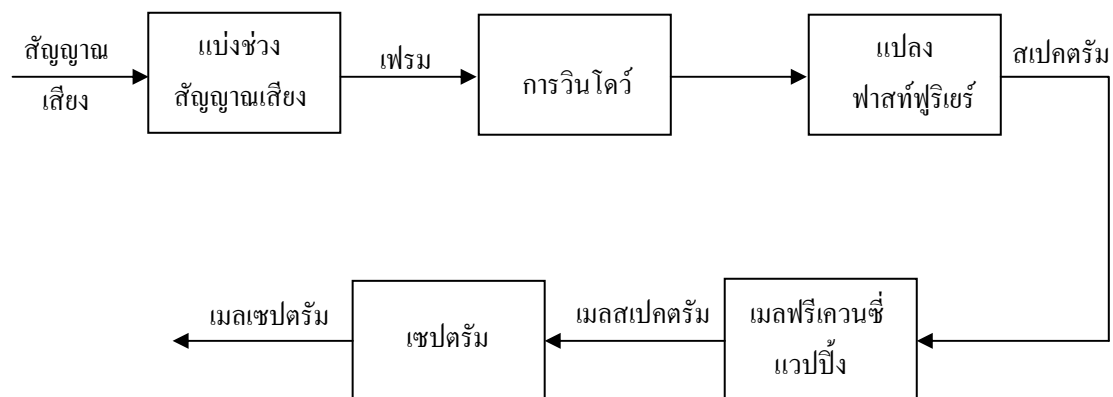
สำหรับปรับสัญญาณให้ราบเรียบ

### 3.4.4 การสกัดค่าลักษณะสำคัญ (Feature extraction)

ในส่วนนี้ จะทำการคำนวณค่าลักษณะสำคัญของสัญญาณเสียงในแต่ละส่วนย่อย ผลลัพธ์อยู่ในรูปแบบของเวกเตอร์ของค่าลักษณะสำคัญ (Feature vector) สำหรับแต่ละส่วนย่อย

### 3.4.5 การประมวลผลของเมลฟรีควเ็นซีเซปตรัมโคเอฟฟิเชียน (Mel-Frequency Cepstrum Coefficients Processor)

แผนภาพโครงสร้างของการประมวลผลเอ็มเอฟซีซีแสดงดังรูปที่ 3.6



รูปที่ 3.6 แสดงโครงสร้างของการประมวลผลเอ็มเอฟซีซี

โดยเราเลือกความถี่ในการสุ่ม 11025 เฮิรตซ์ ซึ่งความถี่นี้ครอบคลุมความถี่ของเสียงพูดมนุษย์ โดยจุดประสงค์ของการประมวลผลเอ็มเอฟซีซีคือการเลียนแบบพฤติกรรมของหูมนุษย์

#### 3.4.5.1 การแบ่งช่วงสัญญาณ (Frame Blocking)

เสียงต้นฉบับจะถูกแบ่งเป็นบล็อก (block) แต่ละบล็อกประกอบด้วย 1024 แซมเปิลและบล็อกที่ติดกันมีช่วงที่ซ้อนทับกัน 512 แซมเปิล โดยบางแซมเปิลที่จุดสิ้นสุดของไฟล์ไม่ได้นำมาใช้ในการคำนวณ

#### 3.4.5.2 การวินโดว์ (Windowing)

แต่ละบล็อกจะถูกวินโดว์เพื่อลดผลของการเปลี่ยนแปลงที่จุดเริ่มต้นและจุดสิ้นสุดของบล็อก เราเลือกใช้หน้าต่างแฮมมิง (Hamming Window) ดังสมการ (3.1)

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad ; n = 0, 1, 2, \dots, N-1 \quad (3.1)$$

โดยฟังก์ชันหน้าต่าง (window function) จะถูกคูณกับเมตริกซ์ (matrix) ที่ได้จากการแบ่งช่วงสัญญาณ

### 3.4.5.3 การแปลงฟาสต์ฟูริเยร์ ( Fast Fourier Transform หรือ FFT )

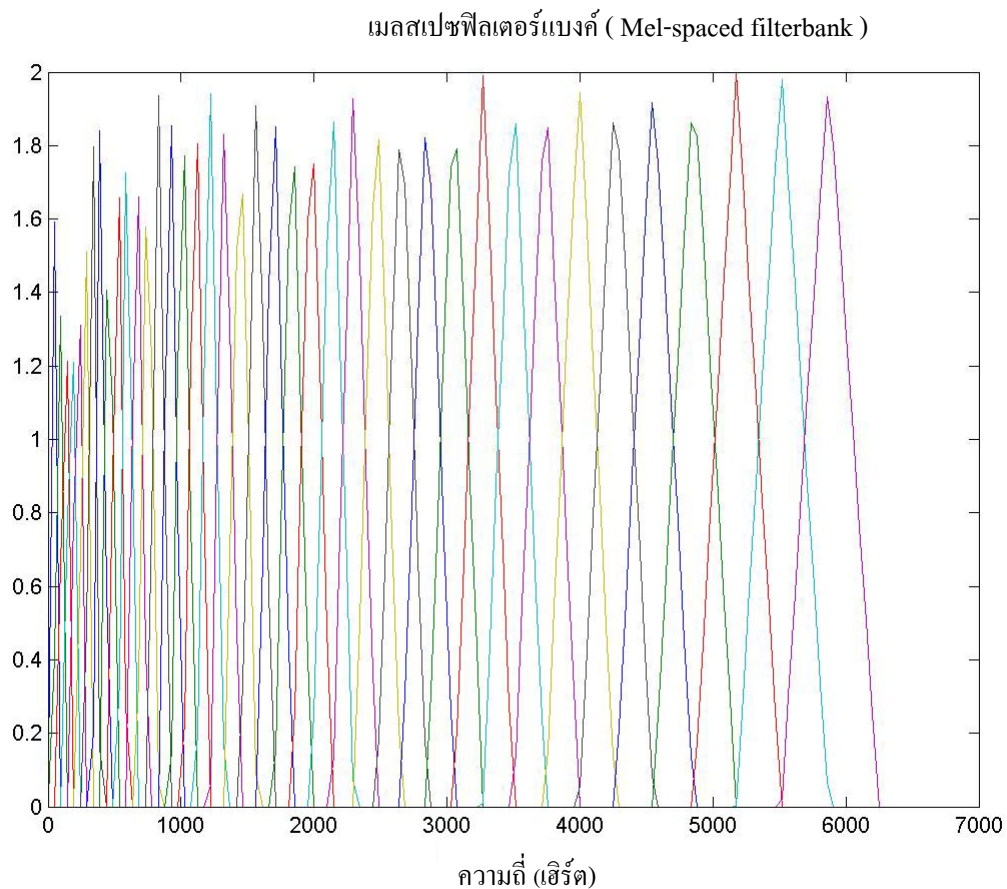
นำบล็อกที่ผ่านการวินโดว์แล้วมาผ่านการแปลงฟาสต์ฟูริเยร์ เพื่อแปลงข้อมูลจากโดเมนเวลาสู่โดเมนความถี่ และสร้างสัมประสิทธิ์สเปกตรัม

### 3.4.5.4 เมลฟรีควเอนซีแวกปีง ( Mel-Frequency Wrapping )

นำการแปลงเมลฟรีควเอนซีมาใช้ในการแปลงแต่ละสเปกตรัมให้อยู่ในรูปแบบเมลสเกล (melscale) โดยค่าที่อ้างอิงคือ ระดับเสียง 1 กิโลเฮิร์ตและ 40 เดซิเบลเหนือค่าขอบเขตการได้ยินของเพอเซปตรัม ( perceptual ) นิยามเป็น 1000 เมล ( mel ) สูตรตามสมการ ( 4.2 ) ใช้ในการคำนวณเมลจากความถี่  $f$  ที่ได้

$$mel(f) = 2595 * \log_{10}(1 + f / 700) \quad (3.2)$$

สเปกตรัมสามารถสร้าง ( simulate ) โดยใช้ฟิลเตอร์แบงก์ที่มีการกระจายตัวด้วยระยะห่างที่สม่ำเสมอบนเมลสเกล ฟิลเตอร์แบงก์ตอบสนองต่อการแบนด์พาสความถี่เป็นรูปสามเหลี่ยม โดยระยะห่างและแบนด์วิดท์จะถูกกำหนดโดยค่าความแตกต่างระหว่างค่าคงที่ของความถี่เมล เราเลือกฟิลเตอร์แบงก์ขนาด 40 ฟิลเตอร์แบงก์ที่เราสร้างขึ้นเป็นดังรูปที่ 3.7



รูปที่ 3.7 แสดงเมลสเปซฟิลเตอร์แบงก์ขนาด 40

### 3.4.5.5 เซปตรัม (Cepstrum)

ใช้ฟังก์ชันลอการิทึม (Log) เพื่อแปลงเมลสเปกตรัมกลับสู่โดเมนเวลา เพื่อหาเมลฟริควเอนซี เซปตรัมโคเอฟฟิเชียนต์เนื่องจากเมลสเปกตรัมโคเอฟฟิเชียนต์เป็นจำนวนจริง เราจึงใช้การแปลง ดิสครีตโคไซน์ (Discrete Cosine Transform หรือ DCT) โดยเอ็มเอฟซีซีคำนวณได้จากสมการ (3.3)

$$\tilde{c}_n = \sum_{k=1}^K (\log \tilde{S}_k) \cos \left[ n \left( k - \frac{1}{2} \right) \frac{\pi}{K} \right] \quad ; n = 1, 2, \dots, K \quad (3.3)$$

ซึ่ง  $\tilde{S}_k, k = 1, 2, \dots, K$  แทนสัมประสิทธิ์เมลสเปกตรัม

เราจะได้เมลฟริควเอนซีเซปตรัมโคเอฟฟิเชียนต์สำหรับเสียงในแต่ละช่วงสัญญาณซึ่งเราเรียกว่า อคูสติกเวกเตอร์ (acoustic vector) ในแถวแรกของเมตริกซ์ที่ได้เราไม่นำมาใช้เพราะเป็น ค่าเฉลี่ยของสัญญาณอินพุตซึ่งเป็นข้อมูลที่มีความสำคัญน้อยมาก ทำให้เราได้เมตริกซ์ขนาด 39 แถว

## 3.5 การรู้จำ (Recognition)

ขั้นตอนนี้ประกอบด้วย 2 หน้าหลัก คือการนำเวกเตอร์ของค่าลักษณะสำคัญของ สัญญาณเสียง ที่อยู่ในชุดอ้างอิงหรือชุดฝึกฝน มาทำการเรียนรู้ เมื่อเรียนรู้แล้วเวกเตอร์ของ สัญญาณเสียงที่ต้องการทดสอบการรู้จำ จะถูกนำเข้ามาเทียบเคียงเพื่อรู้จำ ขั้นตอนในการเรียนรู้ นั้น ขึ้นอยู่กับวิธีการรู้จำของระบบนั้นๆ บางวิธีก็เพียงแค่เก็บข้อมูลชุดเรียนรู้ไปเปรียบเทียบกับข้อมูล ชุดทดสอบเท่านั้น เช่น วิธีการรู้จำแบบหาค่าระยะห่างยูคลิดีเนียน (Euclidean distance) วิธีไดนามิก ไทม์วาร์ปิง (Dynamic time warping: DTW) เป็นต้น ในขณะที่บางวิธี จะนำข้อมูลชุดเรียนรู้ไป แปลงเป็นค่าอ้างอิงที่ต้องการ เช่น โครงข่ายประสาทเทียม (Artificial neural networks: ANN) จะนำ ข้อมูลชุดเรียนรู้ไปผ่านโครงข่ายที่สร้างขึ้น เพื่อจดจำรูปแบบ และเก็บเป็นค่าน้ำหนัก (Weight) แทน วิธีควอนไทซ์แบบเวกเตอร์ (Vector quantization: VQ) ซึ่งจะแทนเวกเตอร์ทั้งหมด ของแต่ละ สัญญาณเสียงอ้างอิงด้วยเวกเตอร์จำนวนไม่มาก หรือการใช้แบบจำลองฮิดเดนมาร์คอฟ (Hidden markov model: HMM) โดยนำข้อมูลชุดฝึกฝนไปผ่านแบบจำลองที่สร้างขึ้นเพื่อจดจำรูปแบบ และ เก็บค่าทางสถิติและค่าความน่าจะเป็นของแต่ละสถานะไว้ เป็นต้น แต่ทั้งหมดจะมีพื้นฐานอยู่ที่การ คำนวณระยะห่างของรูปแบบที่จะรู้จำและนำค่าระยะห่างที่ได้ไปใช้รู้จำตามแต่ละวิธีนั้นๆ

การเลือกใช้วิธีการรู้จำ ขึ้นอยู่กับข้อกำหนดของงาน เช่น วิธี DTW และ ANN เหมาะสมกับ ระบบแบบกำหนดคำพูดตายตัว ในขณะที่วิธี VQ และ HMM จะเหมาะสมกับระบบงานที่เป็นแบบ ไม่กำหนดคำพูดมากกว่า อีกทั้งยังต้องพิจารณาข้อดี-ข้อเสียของแต่ละวิธีด้วย

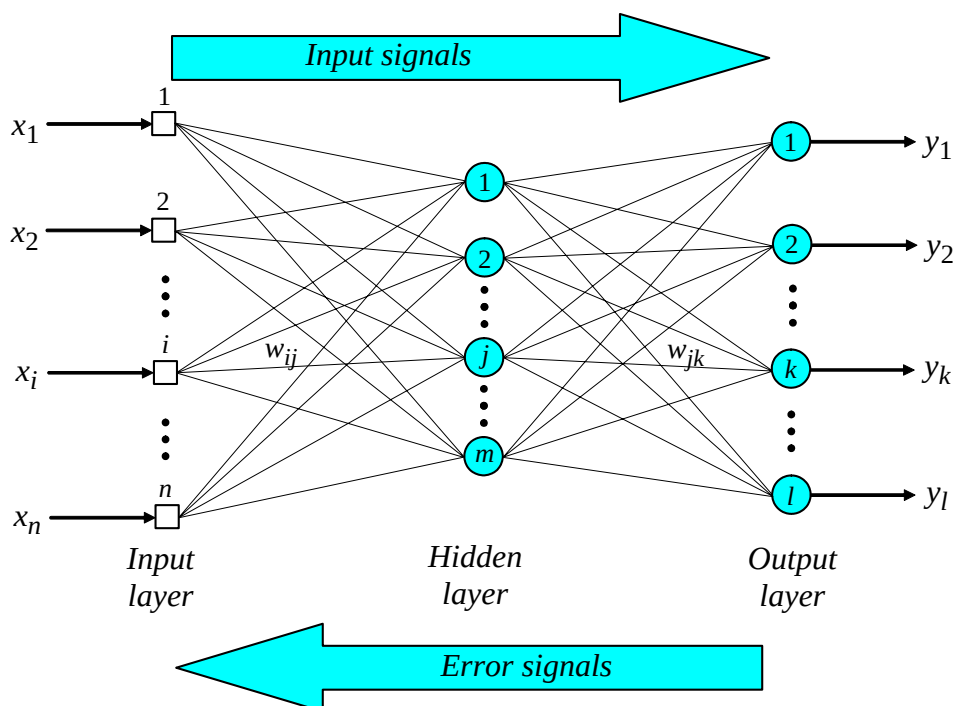


ในปริณยานิพนธ์ ได้ทำการออกแบบส่วนรู้จำโดยใช้วิธีนิวรอลเน็ตเวิร์คแบบแบ็คพรอพพาเกชัน (Backpropagation) ในการตัดสินใจเสียงที่เข้ามาเป็นเสียงของคำสั่ง 10 คำสั่ง ขั้นตอนของการฝึกสอน จะใช้เสียงในการฝึกสอนของผู้พูด 20 คน เสียงละ 1 ครั้ง 10 เสียง รวม 200 เสียง โดยจะใช้รายละเอียดโมดูลของนิวรอลเน็ตเวิร์ค ดังนี้

|                |   |     |
|----------------|---|-----|
| INPUT NEURONS  | = | 624 |
| HIDDEN NEURONS | = | 20  |
| OUTPUT NEURONS | = | 10  |
| LEARNING RATE  | = | 0.1 |

### 3.5.1 กระบวนการ Back – propagation โดยใช้ Neural network

เป็นการเรียนรู้ค่าพารามิเตอร์ของอินพุตแต่ละตัวโดยใช้นิวรอลเน็ตเวิร์ค โดยเปรียบเทียบกับรูปแบบเอาต์พุตที่กำหนดตายตัว ระหว่างที่ค่าที่ได้จากการเรียนค่าพารามิเตอร์ของอินพุตยังไม่ตรงกับรูปแบบเอาต์พุตที่ต้องการ ระบบจะมีการปรับค่าน้ำหนักในนิวรอลเน็ตเวิร์คเพื่อให้ได้ค่าที่ตรงกับรูปแบบเอาต์พุตที่ต้องการมากที่สุด



รูปที่ 3.8 แสดง Back – propagation ขนาด 3 เลเยอร์

### 3.5.1.1 การกำหนดค่าเริ่มต้น

ทำการกำหนดค่าน้ำหนักและค่าเทรโซลด์ในเน็ตเวิร์ก โดยสุ่มความถี่ค่าให้กระในช่วงตามสมการข้างล่าง

$$\left( -\frac{2.4}{F_i}, +\frac{2.4}{F_i} \right) \quad (3.4)$$

โดย  $F_i$  คือจำนวนอินพุตทั้งหมดในเน็ตเวิร์ก

### 3.5.1.2 การคำนวณค่า

คำนวณค่าในระบบ back – propagation โดยค่าอินพุต  $x_1(p), x_2(p), \dots, x_n(p)$  จะได้ค่าเป็น  $yd,1(p), yd,2(p), \dots, yd,n(p)$  ที่เลเยอร์ถัดไป

(1) คำนวณค่าเอาต์พุตที่ได้จริงใน hidden layer ดังสมการ

$$y_j(p) = \text{sigmoid} \left[ \sum_{i=1}^n x_i(p) \cdot w_{ij}(p) - \theta_j \right] \quad (3.5)$$

โดย  $n$  คือ จำนวนอินพุตทั้งหมดของนิวรอนใน hidden layer โดยใช้สมการของซิกมอยด์

(2) คำนวณค่าเอาต์พุตที่ได้จริงใน output layer ดังสมการ

$$y_k(p) = \text{sigmoid} \left[ \sum_{j=1}^m x_{jk}(p) \cdot w_{jk}(p) - \theta_k \right] \quad (3.6)$$

โดย  $m$  คือ จำนวนอินพุตนิวรอนใน output layer

### 3.5.1.3 การปรับปรุงค่าน้ำหนัก

การปรับปรุงค่าน้ำหนักในนิวรอนเน็ตเวิร์กจะมีความสัมพันธ์กับค่าความแตกต่างของเอาต์พุตโดยแบ่งเป็น

(1) คำนวณค่าความคลาดให้ค่าน้ำหนักเปลี่ยนแปลงเข้าสู่จุดดีที่สุด ใน output layer จากสมการ

$$\delta_k(p) = y_k(p) \cdot [1 - y_k(p)] \cdot e_k(p) \quad (3.7)$$

โดย  $e_k(p) = y_{d,k}(p) - y_k(p)$  (3.8)

ต่อจากนั้น คำนวณค่าน้ำหนักที่ถูกต้อง จากสมการ

$$\Delta w_{jk}(p) = \alpha \cdot y_j(p) \cdot \delta_k(p) \quad (3.9)$$

ปรับปรุงค่าน้ำหนักที่ควรจะเป็น จากสมการ

$$w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p) \quad (3.10)$$

(2) คำนวณค่าความคลาดให้น้ำหนักเปลี่ยนแปลงเข้าสู่จุดที่ดีที่สุด ใน hidden layer จากสมการ

$$\delta_j(p) = y_j(p) \cdot [1 - y_j(p)] \cdot \sum_{k=1}^l \delta_k(p) w_{jk}(p) \quad (3.11)$$

ต่อจากนั้น คำนวณค่าน้ำหนักที่ถูกต้อง จากสมการ

$$\Delta w_{ij}(p) = \alpha \cdot x_i(p) \cdot \delta_j(p) \quad (3.12)$$

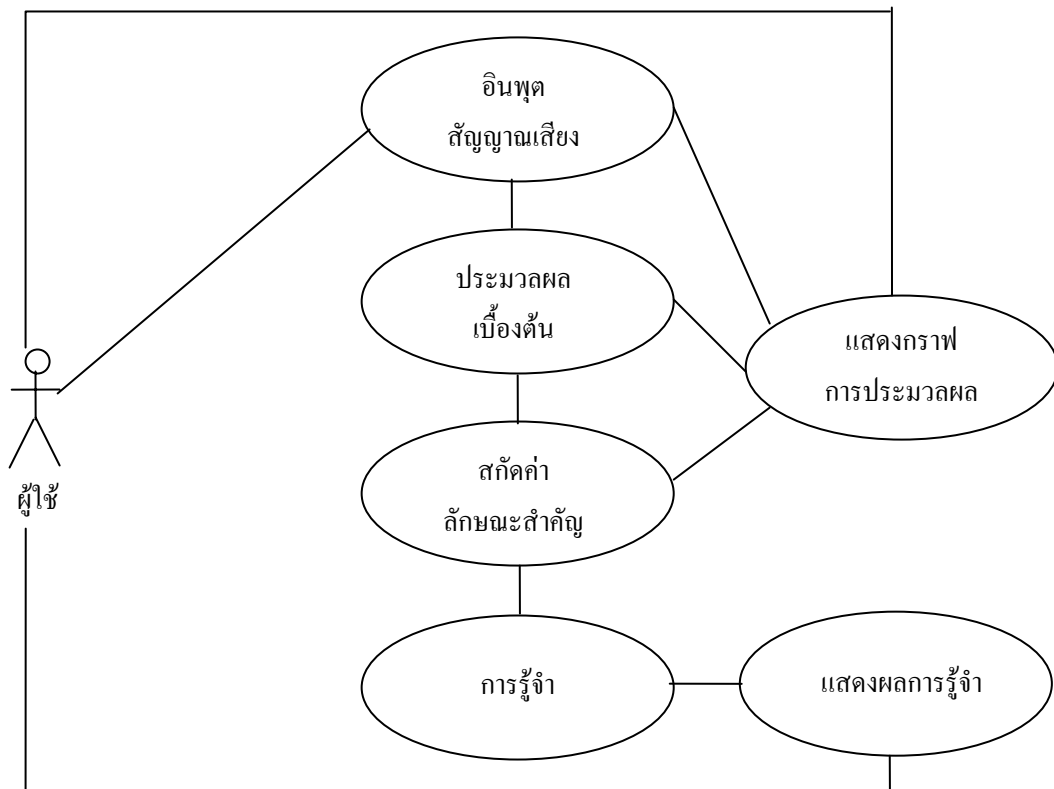
ปรับปรุงค่าน้ำหนักที่ควรจะเป็น จากสมการ

$$w_{ij}(p+1) = w_{ij}(p) + \Delta w_{ij}(p) \quad (3.13)$$

### 3.5.1.3 การทำซ้ำ

จากจากเสร็จสิ้นการปรับปรุงค่าน้ำหนักทั้งหมด ระบบจะกลับไปคำนวณที่ขั้น 3.5.1.2 โดยจะทำซ้ำไปเรื่อยจนกระทั่งค่า error ที่ได้จะเข้ามาสู่จุดที่เราพอใจ

## 3.6 UML Diagram ระบบรู้จำเสียง



### 3.7 Sequence Diagram

